第十四届“恩智浦”杯全国大学生
智能汽车竞赛

技 术 报 告

无人驾驶汽车的研究

学　　校：淮南联合大学

队伍名称：室外光电组

参赛队员：马艳伟 王顺 杨坤 沈至庆 王浩

带队教师：李锦鹏 程双龙

关于技术报告和研究论文使用授权的说明

本人完全了解第 14 届“恩智浦”杯全国大学生智能汽车竞赛关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会和恩智浦半导体公司可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：

马艳伟　王顺 杨坤

沈至庆　王浩

带队教师签名：

李锦鹏 程双龙

日期：2019.8.10

目录

1.引言.....2
1.1 比赛介绍.....2
1.2 方案介绍.....2
2.机械结构.....4
2.1 机械结构介绍.....4
3.硬件电路设计.....5
3.1 硬件电路介绍.....5
3.2 单片机设计电路.....5
3.3 电源接入接口电路.....6
3.4 电源设计电路.....6
3.5 USB 转串口电路.....7
3.6 USB 转串口电路（电脑---雷达）.....7
3.7 接口电路.....8
4.智能车软件设计.....10
4.1 软件设计思路.....10
4.2 AMCL 算法介绍.....10
4.3 单片机程序设计思路.....17
4.4 单片机程序.....18
5.结论.....23
5.1 存在的问题.....23
5.2 体会.....23

1.引言

1.1 比赛介绍

为加强大学生实践、创新能力和团队精神的培养，促进高等教育教学改革，受教育部高等教育司委托，由教育部高等学校自动化专业教学指导分委员会（以下简称自动化分教指委）主办全国大学生智能汽车竞赛。该竞赛是以智能汽车为研究对象的创意性科技竞赛，是面向全国大学生的一种具有探索性工程实践活动，是教育部倡导的大学生科技竞赛之一。该竞赛以“立足培养，重在参与，鼓励探索，追求卓越”为指导思想，旨在促进高等学校素质教育，培养大学生的综合知识运用能力、基本工程实践能力和创新意识，激发大学生从事科学研究与探索的兴趣和潜能，倡导理论联系实际、求真务实的学风和团队协作的人文精神，为优秀人才的脱颖而出创造条件

全国大学生“恩智浦”杯智能汽车竞赛由竞赛秘书处设计、规范标准硬软件技术平台，竞赛过程包括理论设计、实际制作、整车调试、现场比赛等环节，要求学生组成团队，协同工作，初步体会一个工程性的研究开发项目从设计到实现的全过程。该竞赛融科学性、趣味性和观赏性为一体，是以迅猛发展、前景广阔的汽车电子为背景，涵盖自动控制、模式识别、传感技术、电子、电气、计算机、机械与汽车等多学科专业的创意性比赛。该竞赛规则透明，评价标准客观，坚持公开、公平、公正的原则，力求向健康、普及、持续的方向发展。

全国大学生“恩智浦”杯智能汽车竞赛是在规定的模型汽车平台上，使用飞思卡尔半导体公司的 8 位、16 位微控制器作为核心控制模块，通过增加道路传感器、电机驱动电路以及编写相应软件，制作一个能够自主识别道路的模型汽车，按照规定路线行进，以完成时间最短者为优胜。因而该竞赛是涵盖了控制、模式识别、传感技术、电子、电气、计算机、机械等多个学科的比赛。

1.2 方案介绍

我们参加的室外光电组以实现无人驾驶为目标，硬件部分主要以主控电脑为核心，外部搭载激光雷达传感器，用来采集整车周围所有的物体相关的距离信息；同时搭载了姿态传感器，用来获得车辆整体的姿态信息和加速度信息。单片机（MC9S08AC16CFGE）系统与主控电脑端通过 USB 转串口连接，单片机系统通过接收 ROS 系统下达的线速度与角速度指令进行运动学方程转换，通过 PWM 控制直流无刷电机和舵机，进而实现对车体的速度与方向的控制。通过路由器

组件局域网，我们可以使用 ssh 远程访问登陆主控电脑，方便操作人员实现对整个系统远程控制。

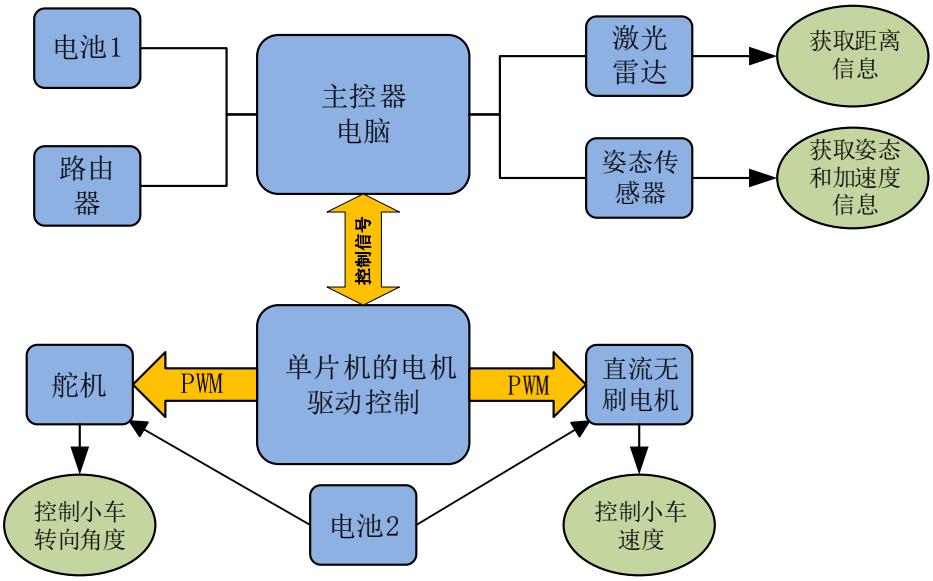


图 1 智能车系统架构图

软件系统部分如图 1 所示，机器人操作系统 ROS 通过外部激光雷达和 IMU 姿态传感器获取无人竞速车周围环境信息与车体的姿态信息，将这些采集到的数据进行处理，通过 SLAM 算法实现对整个环境的地图构建，有了地图之后，我们就可以通过 AMCL 等算法实现地图上的路径规划和自主导航。

2.机械结构

2.1 机械结构介绍

本组别比赛所用车模为钢铁侠公司提供的 G 车模，如图 2 所示，所用电机为 ART-M1-K2150 有感无刷电机，所用舵机为 SG995 金属齿轮舵机。并配备镭神 LS01G 激光雷达，用来构建地图并用来检测周围障碍物，IMU-02A 姿态传感器则用来拟合小车前进的路径。



图 2 G 车模实物图

3. 硬件电路设计

3.1 硬件电路介绍

因为本届比赛是第一届无人驾驶组比赛，故降低了难度，要求只需要设计主控板就可以，上位机使用了微型电脑，并利用无线路由进行电脑与客户端的连接，用来调试赛车。我们的主控芯片选择了恩智浦的 MC9S08AC16CFGE 处理器

此处理器具有 128Kb 的闪存、3 个定时器/脉宽调制器（PWM）、16 通道 10 位模数转换器（ADC），从而拓展了 8 位架构的界限，并提供高达 20MHz 的总线频率。并具有升级版串行通信接口（SCI），能够用来提供单片机与雷达，与电脑之间的串行通信，3 路定时器/PWM（TPM），可以方便的生成多路 PWM 波用来控制舵机的打舵角度和电机的速度。

3.2 单片机设计电路

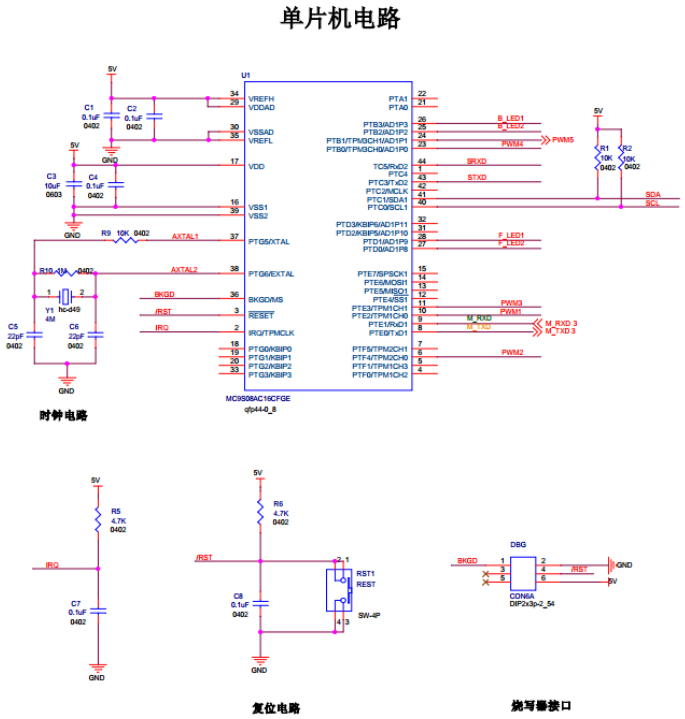


图 3 单片机设计电路

单片机电路的设计如图 3 所示，其实就是单片机最小系统板的设计，主要包含复位电路，时钟电路-用来产生单片机工作所需要的节拍，所选晶振为 4Mhz，还有程序下载接口。

3.3 电源接入接口电路

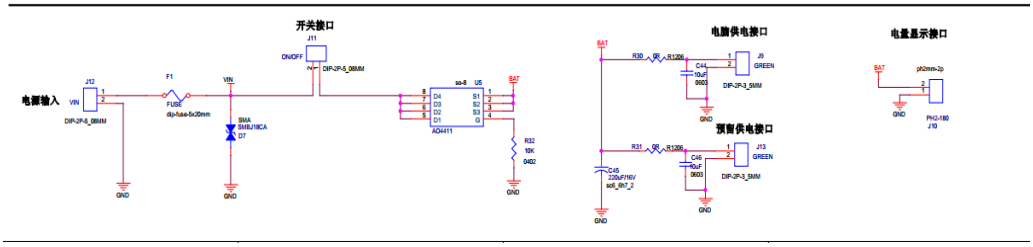


图 4 电源接入电路

电源接入电路如图 4 所示，为了增加系统的安全性和可靠性，电源接入部分增加了一个稳压二极管，当电源输入电压过高时，二极管处于反向击穿状态，保持电压不至于波动过大，同时为了防止电源反接，利用了 MOS 管的单向导电性，使用 AD4411 MOS 管保证了电源在反接状态的时候，MOS 管处于截止状态，保证后面的电子元器件不会被损坏。

3.4 电源设计电路

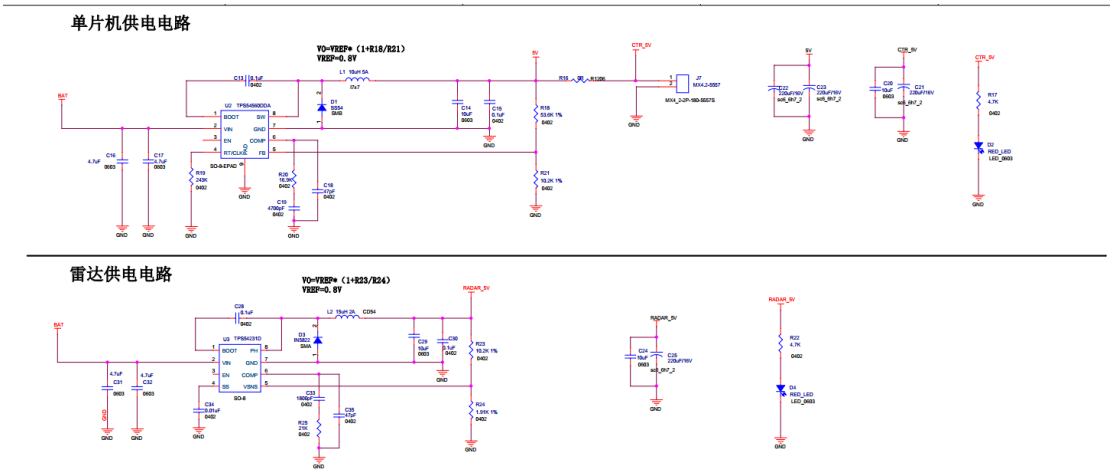


图 5 单片机路由器供电电路

本电路设计中，单片机需要 5V 供电，舵机需要 4.8-6v 的供电，为了简化电路设计，将舵机的供电电压也设计成 5V,统一电压等级。这样就可以使用同一电源芯片。因为所用电池的电压标称为 12v，充满电的时候可以达到 13v 以上，变成 5V 电压，压差较大，所以在众多的电源芯片中选择了 TPS54560DDA。该款芯片最大输入电压可达到 60V，最大输出电流可达到 5A,非常适合无人驾驶赛项。我们使用三个电源芯片，设计了三组能够生成 5V 的电路，如图 5，图 6 所示。一路用来给单片机与路由器供电，一路用来给雷达供电，一路用来预留

USB 口供电。

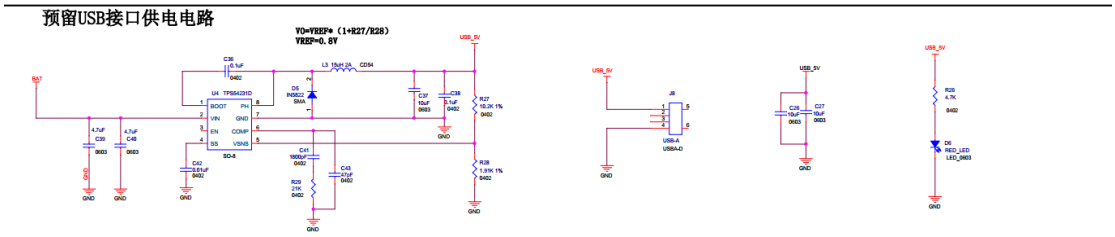


图 6 预留 USB 供电电路

3.5 USB 转串口电路

在本设计中，微电脑下达舵机打角和控制电机速度所需要的 PWM 的指令，将数据传递给单片机，单片机接受到指令后，负责输出两路 PWM 控制舵机的角度和电机的速度。因为微电脑的接口常用的是 USB 接口，而单片机所用的一般的是串行通信口，所以我们需要进行将通信格式进行转换，将 USB 信号转为串口信号。为了实现这两种不同格式信号的传递，我们选用了 CH340G 芯片。CH340G 是一个 USB 总线的转接芯片，能够实现 USB 转串口、USB 转 IrDA 红外或者 USB 转打印口。具体电路如图 7 所示。

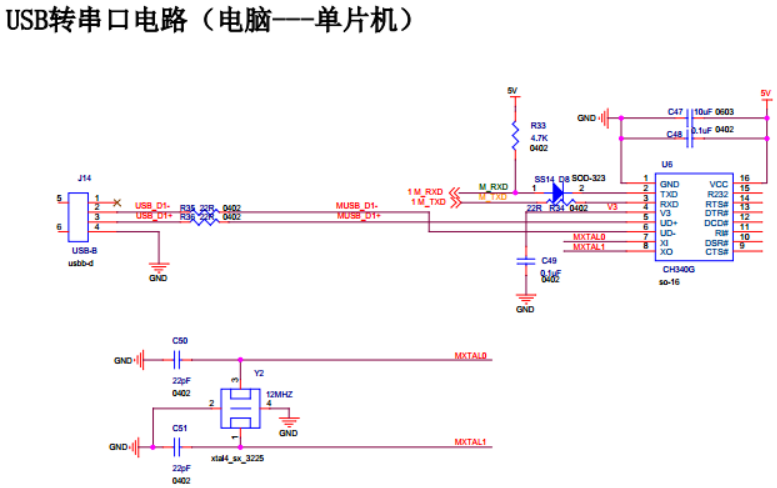


图 7 USB 转串口电路

3.6 USB 转串口电路（电脑---雷达）

雷达输出扫描到的障碍物信息是通过 UART 串口输出给电脑的，所以同样也需要将信息转为 USB 信息，因为雷达输出的信息波特率较高，而雷达输出的信息对于小车避障和前行又至关重要所以我们选择了性能更加优越稳定的 FT232RL 芯片来实现 USB 与串口信息的转换传递。电路设计如图 8 所示。

USB转串口电路（电脑——雷达）

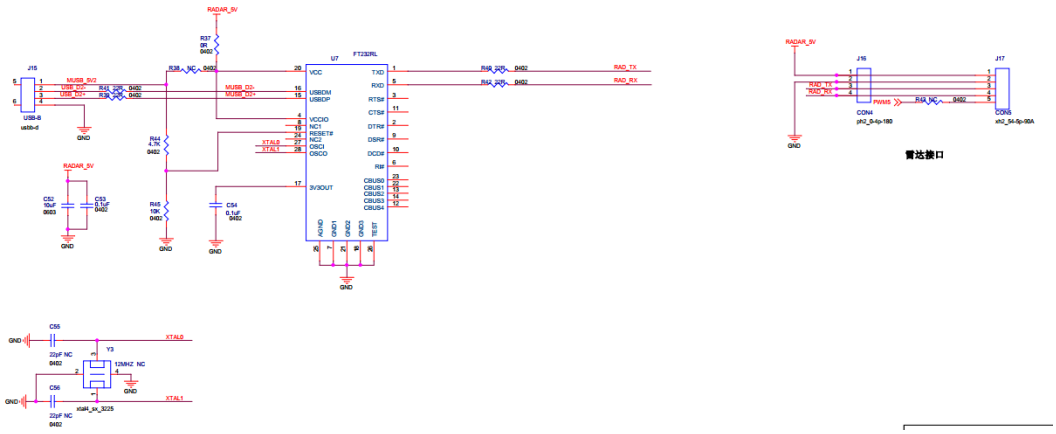


图 8 USB 转串口电路（雷达）

3.7 接口电路

电路中设计了 4 个预留灯接口用来调试的时候指示用，同时留出了舵机控制接口与电调控制接口。如图 9、图 10 所示。

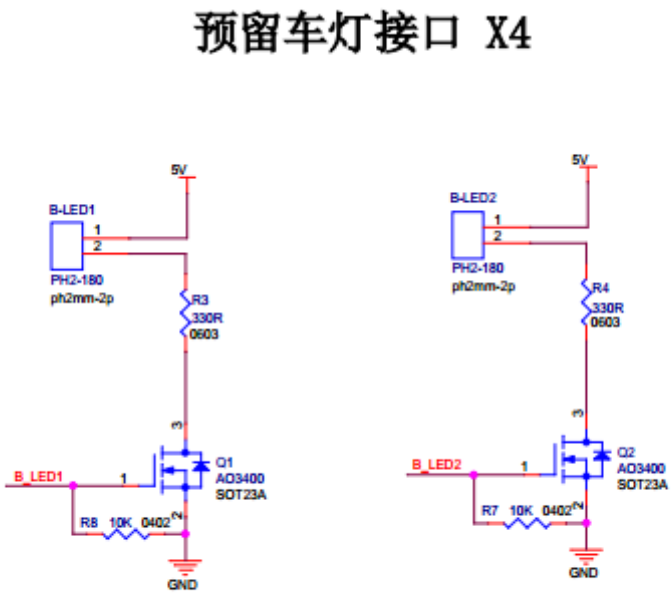


图 9 预留车灯接口

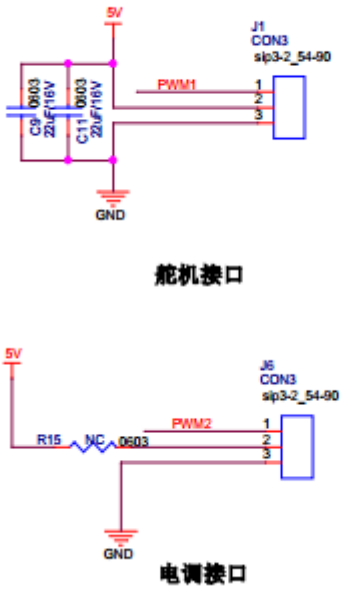

5V
J1
CON3
sip3-2_54-90
PWM1
1
2
3
C9
0603
22uF/16V
C11
0603
22uF/16V
GND
舵机接口
5V
J6
CON3
sip3-2_54-90
PWM2
1
2
3
R15
NC
0603
GND
电调接口

图 10 舵机与电调接口

4. 智能车软件设计

4.1 软件设计思路

机器人操作系统 ROS 通过外部激光雷达和 IMU 姿态传感器获取无人竞速车周围环境信息与车体的姿态信息，将这些采集到的数据进行处理，通过 SLAM 算法实现对整个环境的地图构建，有了地图之后，我们就可以通过 AMCL 等算法实现地图上的路径规划和自主导航。然后通过电脑下发舵机的大脚指令与电机速度控制指令。所以编程主要有上位机电脑编程，和下位机单片机编程。

SLAM (simultaneous localization and mapping),也称为 CML (Concurrent Mapping and Localization)，即时定位与地图构建，或并发建图与定位。问题可以描述为：将一个机器人放入未知环境中的未知位置，是否有办法让机器人一边移动一边逐步描绘出此环境完全的地图，所谓完全的地图是指不受障碍行进到房间可进入的每个角落。

4.2 AMCL 算法介绍

在上位机电脑中设计了 AMCL 算法，AMCL 算法是在实现地图的基础上的一种算法，能够实现小车路径规划和导航。主要代码如下：

```
Panels:
  - Class: rviz/Displays
    Help Height: 78
    Name: Displays
    Property Tree Widget:
      Expanded:
        - /Global Options1
        - /Imu1/Box properties1
      Splitter Ratio: 0.5
    Tree Height: 528
  - Class: rviz/Selection
    Name: Selection
  - Class: rviz/Tool Properties
    Expanded:
      - /2D Pose Estimate1
      - /2D Nav Goal1
      - /Publish Point1
    Name: Tool Properties
    Splitter Ratio: 0.588679016
  - Class: rviz/Views
    Expanded:
```

```
      - /Current View1
    Name: Views
    Splitter Ratio: 0.5
  - Class: rviz/Time
    Experimental: false
    Name: Time
    SyncMode: 0
    SyncSource: LaserScan
Visualization Manager:
  Class: ""
  Displays:
    - Alpha: 0.5
      Cell Size: 1
      Class: rviz/Grid
      Color: 160; 160; 164
      Enabled: true
      Line Style:
        Line Width: 0.0299999993
        Value: Lines
      Name: Grid
      Normal Cell Count: 0
      Offset:
        X: 0
        Y: 0
        Z: 0
      Plane: XY
      Plane Cell Count: 100
      Reference Frame: <Fixed Frame>
      Value: true
    - Alpha: 0.699999988
      Class: rviz/Map
      Color Scheme: map
      Draw Behind: false
      Enabled: true
      Name: Map
      Topic: /map
      Unreliable: false
      Use Timestamp: false
      Value: true
    - Alpha: 0.200000003
      Class: rviz/Map
      Color Scheme: costmap
      Draw Behind: false
      Enabled: true
```

```
    Name: global_costmap
    Topic: /move_base/global_costmap/costmap
    Unreliable: false
    Use Timestamp: false
    Value: true
  - Alpha: 1
    Autocompute Intensity Bounds: true
    Autocompute Value Bounds:
      Max Value: 10
      Min Value: -10
      Value: true
    Axis: Z
    Channel Name: intensity
    Class: rviz/LaserScan
    Color: 255; 255; 255
    Color Transformer: Intensity
    Decay Time: 0
    Enabled: true
    Invert Rainbow: false
    Max Color: 255; 255; 255
    Max Intensity: 909
    Min Color: 0; 0; 0
    Min Intensity: 2
    Name: LaserScan
    Position Transformer: XYZ
    Queue Size: 10
    Selectable: true
    Size (Pixels): 3
    Size (m): 0.0500000007
    Style: Flat Squares
    Topic: /scan
    Unreliable: false
    Use Fixed Frame: true
    Use rainbow: true
    Value: true
  - Class: rviz/TF
    Enabled: true
    Frame Timeout: 15
    Frames:
      All Enabled: true
      IMU_link:
        Value: true
      base_footprint:
        Value: true
```

```
      base_link:
        Value: true
      laser:
        Value: true
      map:
        Value: true
      odom:
        Value: true
    Marker Scale: 1
    Name: TF
    Show Arrows: true
    Show Axes: true
    Show Names: true
    Tree:
      map:
        odom:
          base_footprint:
            base_link:
              IMU_link:
                {}
              laser:
                {}
    Update Interval: 0
    Value: true
  - Alpha: 1
    Buffer Length: 1
    Class: rviz/Path
    Color: 25; 255; 0
    Enabled: true
    Head Diameter: 0.300000012
    Head Length: 0.200000003
    Length: 0.300000012
    Line Style: Lines
    Line Width: 0.0299999993
    Name: Path
    Offset:
      X: 0
      Y: 0
      Z: 0
    Pose Color: 255; 85; 255
    Pose Style: None
    Radius: 0.0299999993
    Shaft Diameter: 0.100000001
    Shaft Length: 0.100000001
```

```
    Topic: /move_base/NavfnROS/plan
    Unreliable: true
    Value: true
  - Angle Tolerance: 0.100000001
    Class: rviz/Odometry
    Covariance:
      Orientation:
        Alpha: 0.5
        Color: 255; 255; 127
        Color Style: Unique
        Frame: Local
        Offset: 1
        Scale: 1
        Value: true
      Position:
        Alpha: 0.300000012
        Color: 204; 51; 204
        Scale: 1
        Value: true
      Value: false
    Enabled: true
    Keep: 100
    Name: Odometry
    Position Tolerance: 0.100000001
    Shape:
      Alpha: 1
      Axes Length: 1
      Axes Radius: 0.100000001
      Color: 255; 25; 0
      Head Length: 0.300000012
      Head Radius: 0.100000001
      Shaft Length: 0.100000001
      Shaft Radius: 0.0199999996
      Value: Arrow
    Topic: /odometry/filtered
    Unreliable: false
    Value: true
  - Acceleration properties:
      Acc. vector alpha: 1
      Acc. vector color: 255; 0; 0
      Acc. vector scale: 1
      Derotate acceleration: true
      Enable acceleration: false
    Axes properties:
```

```
      Axes scale: 1
      Enable axes: true
    Box properties:
      Box alpha: 1
      Box color: 255; 0; 0
      Enable box: false
      x_scale: 0.400000006
      y_scale: 0.300000012
      z_scale: 0.300000012
    Class: rviz_imu_plugin/Imu
    Enabled: true
    Name: Imu
    Topic: /imu_data
    Unreliable: false
    Value: true
    fixed_frame_orientation: true
Enabled: true
Global Options:
  Background Color: 48; 48; 48
  Default Light: true
  Fixed Frame: map
  Frame Rate: 30
Name: root
Tools:
  - Class: rviz/Interact
    Hide Inactive Objects: true
  - Class: rviz/MoveCamera
  - Class: rviz/Select
  - Class: rviz/FocusCamera
  - Class: rviz/Measure
  - Class: rviz/SetInitialPose
    Topic: /initialpose
  - Class: rviz/SetGoal
    Topic: /move_base_simple/goal
  - Class: rviz/PublishPoint
    Single click: true
    Topic: /clicked_point
Value: true
Views:
  Current:
    Class: rviz/Orbit
    Distance: 17.7873268
    Enable Stereo Rendering:
      Stereo Eye Separation: 0.0599999987
```

```
      Stereo Focal Distance: 1
      Swap Stereo Eyes: false
      Value: false
    Focal Point:
      X: 1.2478348
      Y: -3.33640909
      Z: 0.00110054587
    Focal Shape Fixed Size: true
    Focal Shape Size: 0.0500000007
    Invert Z Axis: false
    Name: Current View
    Near Clip Distance: 0.00999999978
    Pitch: 1.56979632
    Target Frame: <Fixed Frame>
    Value: Orbit (rviz)
    Yaw: 2.89539313
  Saved: ~
Window Geometry:
  Displays:
    collapsed: false
  Height: 809
  Hide Left Dock: false
  Hide Right Dock: true
  QMainWindow State:
000000ff00000000fd0000000400000000000016a0000029ffc020000000bfb0000001
200530065006c00650063007400690006f006e00000001e10000009b0000006100ffffff
b0000001e0054006f006f006c002000500072006f007000650072007400690065007300
2000001ed000001df00000185000000a3fb0000001200560069006500770073002000 5
4006f006f02000001df000002110000018500000122fb000000200054006f006f006c00
200050007200 6f007000650072007400690065007300320300000288000001 1d00000
2210000017afb0000010004400690073007000 6c006100790073010000002800000 2
9f000000d700ffffffb0000002000730065006c006500630074006900 6f006e00200062
007500660066006500720200000138000000aa0000023a00000294fb0000001400570
069006400650053007400650072006500 6f02000000e6000000d2000003ee0000030bf
b0000000c004b0069006e006500630074020000018600000106000003 0c00000261fb
0000000c00430061006d006500720061000000001d1000000b5000000000000000fb0
000000a0049006d006100670065000000000ffffffff0000000000000000fb0000000a0
049006d006100670065010000 02ab000001130000000000000000000000010000 1a3
0000025efc0200000004fb0000000a0049006d006100670065000000002800000 25e00
00000000000000fb0000001e0054006f006f006c002000500072006f00700065007200
740069006500730100000041000000780000000000000000fb0000000a00560069006
500770073000000002800000 25e000000ad00ffffffb0000001200530065006c006500
6300740069006f006e010000025a000000b2000000000000000000000002000004900
00000a9fc0100000001fb0000000a00560069006500770073030000004e00000080000
```

002e10000019700000003000005160000003efc0100000002fb0000000800540069006d00650100000000000000516000003000fffffffb0000000800540069006d00650100000000000000450000000000000000000000003a60000029f000000040000000400000008000000008fc0000000100000002000000010000000a0054006f006f006c00730100000000ffffffff0000000000000000

Selection:
collapsed: false
Time:
collapsed: false
Tool Properties:
collapsed: false
Views:
collapsed: false
Width: 1302
X: 612
Y: 47

其它还编写了一些小车的控制程序，例如小车的速度控制程序，舵机打舵控制程序，如何避开障碍物程序等，由于篇幅有限这里不在赘述。

4.3 单片机程序设计思路

单片机的任务很简单只需要按照上位机电脑发来的指令进行舵机打舵控制和电机速度控制即可，舵机和电机速度控制都是单片机通过输出 PWM 来控制的。单片机设计成只需要根据上位机的指令来输出两路的 PWM 波来分别控制电调和舵机，输出 PWM 都为 50HZ，有效占空比区间为 2.5%~12.5%，分别对应电机反向最高速，到正向最高速，及舵机的-90~90 度。

上位机与下位机的通信格式被定义为指令长度固定为 7 字节，占空比用 16 位数据表示，含义为占空比*1000。例：12500 表示占空比为 12.5%。数据使用大端对齐。单片机接收到指令以后会返回当前 PWM 占空比的信息。

固定帧头为 0xAA， 后面 4 字节为数据，1 字节的校验位（check），1 字节的帧尾，校验使用和校验只校验 4 字节的数据。指令格式表 1 所示。

注：check=电机 PWM_H+电机 PWM_L+舵机 PWM_H+舵机 PWM_L

1	2	3	4	5	6	7
0xAA	电机 PWM_H	电机 PWM_L	舵机 PWM_H	舵机 PWM_L	Check	0x55

表 1 电脑下发指令格式

当接收到正确的指令后，单片机会返回当前电机和舵机的 PWM 占空比。指令和下发相同。指令格式如表 2 所示。

1	2	3	4	5	6	7
0xAA	电机 PWM_H	电机 PWM_L	舵机 PWM_H	舵机 PWM_L	Check	0x55

表 2 单片机上传指令格式

4.4 单片机程序

单片机主要程序如下

```
void ICGInit(void)
{

//    /*使用内部时钟*/
//    ICGC1 = 0x08;  //使用内部时钟  +FLL  内部时钟为243k左右P=64    /7*64输入到FLL
//    ICGC2 = 0x60;  //FLL 16倍频    不分频
//    ICGTRM = 0xB6;    //微调内部时钟的频率  默认0x80  使FLL输入信号为调整为2M(1.9996)
//    //主控时钟为32M BUS时钟为16M

      /*使用外部时钟*/
      ICGC1 = 0xF8;  //使用4M晶振  +FLL
      ICGC2 = 0x20;  //FLL 8倍频    不分频
      //主控时钟为32M BUS时钟为16M

}
/*****************************************************************/
/*                          串口接收函数                          */
/*****************************************************************/
void INIT_SCI1(void)
{
   SCI1BDH = 0x00;       //1停止位
   SCI1BDL = 0x1A;           //设置SCI波特率为38400    波特率
=busclk/16/26=38400    busclk约=16MHz
   SCI1C1 = 0x00;            //设置SCI为正常模式，八位数据位，无奇偶校验
   SCI1C2 = 0x2c;            //允许接收和发送数据，允许接收中断功能
}
/*****************************************************************/
/*                          初始化TPM                            */
/*****************************************************************/
void INIT_TPM1(void)
{
   TPM1SC=0x0B;//TPM1时钟源为总线时钟；分频系数为8；溢出中断禁止
                    //总线频率约为8.008,经过分频之后为2000000Hz
```

```
  TPM1MOD = 40000-1;  //PWM波的周期为20ms
  TPM1C0SC = 0X28;    //工作模式为边缘对齐PWM输出，高有效，禁止中断；
  TPM1C0V = 3000;     //有效部分占7.5%，占空比为7.5%
}

void INIT_TPM2(void)
{
  TPM2SC=0x0B;//TPM1时钟源为总线时钟；分频系数为4；溢出中断禁止
                  //总线频率约为8.008,经过分频之后为2000000Hz
  TPM2MOD = 40000-1;  //PWM波的周期为20ms
  TPM2C0SC = 0X28;    //工作模式为边缘对齐PWM输出，高有效，禁止中断；
  TPM2C0V = 3000;     //有效部分占7.5%，占空比为7.5%
}

void INIT_TPM3(void)
{
  TPM3SC=0x0B;//TPM1时钟源为总线时钟；分频系数为4；溢出中断禁止
                  //总线频率约为8.008,经过分频之后为2000000Hz
  TPM3MOD = 40000-1;  //PWM波的周期为20ms
  TPM3C0SC = 0X44;    //输入捕获  上升沿捕获  使能中断；
}

/*************************************************************/
/*                          延时函数                         */
/*************************************************************/
void delay(unsigned int n)
{
  unsigned int i,j;
  for(j=0;j<n;j++)
  for(i=0;i<15000;i++)
  ;
}

/*************************************************************/
/*                          指令应答函数                     */
/*************************************************************/
void SCI_Reply(void)
{
    uint8 tmp=0,sum=0;
    SCI1_send(0xAA);

    tmp=TPM2C0V>>1;
    sum=tmp;
    SCI1_send(tmp);
```

```
    tmp=TPM2C0V>>9;
    sum+=tmp;
    SCI1_send(tmp);

    tmp=TPM1C0V>>1;
    sum+=tmp;
    SCI1_send(tmp);

    tmp=TPM1C0V>>9;
    sum+=tmp;
    SCI1_send(tmp);

    SCI1_send(sum);
    SCI1_send(0x55);

}

    if(cmdm<=2500&&cmdm>=500)
        TPM2C0V=2*cmdm;      //设置舵机PWM

    if(cmds<=2500&&cmds>=500)
        TPM1C0V=2*cmds;//设置电调PWM

//              delay(1);
                SCI_Reply();
}

void main(void) {
    int tmp=0;

    DisableInterrupts;
    SOPT=SOPT&0X7F;     //将COPE位强制置为0，其它位不变，关闭看门狗
    ICGInit();
//  PTGDD=0x08;
//  PTGD=0x08;
    INIT_TPM1();
    INIT_TPM2();
//  INIT_TPM3();

    INIT_SCI1();
    EnableInterrupts;

    init_queue(&gU1RxQ);
```

```
    SCI1_send('s');
    SCI1_send('t');
    SCI1_send('a');
    SCI1_send('r');
    SCI1_send('t');
    SCI1_send('\r');
    SCI1_send('\n');




    for(;;) {
        if(get_queue_size(cmdQ)<7){

        }else if(get_queue_size(cmdQ)>6){     //出现队列长度不够的时候也进入
了 很奇怪
            tmp=get_queue_size(cmdQ);
            Parse_Command();
        }
//        delay(10);

    }

}


/*************************************************************/
/*                          串口中断接收函数                 */
/*************************************************************/
 interrupt VectorNumber_Vsci1rx void receivedata(void)
{
    enqueue(&gU1RxQ, SCI1_receive());
}

// interrupt VectorNumber_Vtpm3ch0 void isr_TPM3ch0(void){
//    if(TPM3C0SC_CH0F == 1)     //判断是否捕获
//    {
//        if(pwtst==0)
{
//            TPM3CNT=0;
//            TPM3C0SC = 0X48;
//            pwtst=1;
//        }
```

```
//          else{
//              pwtcnt=TPM3C0V;
//              TPM3C0SC = 0X44;
//              pwtst=0;
//          }
//
//          TPM3C0SC_CH0F = 0;          //清除捕获标志位
//      }
// }
```

5. 结论

5.1 存在的问题

无人驾驶赛项难度较大，目前上层算法我们理解还不够透彻，只能微调一些参数，大部分精力都放在了下位机单片机的编程上，我们在设计的过程中，相对于其他赛项，雷达只相当于一个传感器，单片机负责控制电机速度和打舵，所以有时候如果地图构建不准，小车容易撞到障碍物或者围栏，为了稳定只能降速处理，大大限制了小车的速度。我们下一步的目标就是把无人驾驶的上位机代码研究透彻，争取从上位机和下位机同时进行设计编程，提高小车的识别精度和速度。

5.2 体会

我们在设计的过程中，过多的注重了程序算法的研究，而忽略了硬件设计，机械结构的调整，导致车速一快在拐弯的时候容易翻车。同时还存在小车在行驶的过程中，上下颠簸的时候，雷达扫描到的信息跟实际路况相比误差很大。下一步我们打算，第一将电路板进一步优化做到主控与驱动合一，减少电路板的体积。第二研究透小车的机械结构，将小车的有些部位加固，增加车体的稳定性。